

Lab04-Amortized&Graph

CS2312-Algorithm and complexity, Xiaofeng Gao, Spring 2026.

* Contact TA Yixuan Cai for any questions. Include both your .pdf and .tex files in the uploaded .rar or .zip file.
* Name:Hengtao Wu Student ID:524031910038 Email: Eternity_w@sjtu.edu.cn

1. (Amortized in special data structure) Consider a special data structure consists of k irrelevant (i.e. elements in different arrays do not have sequence constraint) non-decreasing arrays $A_0, A_1, \dots, A_{k-1}$. The size of array A_i is 2^i . At any time, any array is either full or empty. Suppose the total number of elements is n . How can we insert an element into such data structure? Please analyze the time cost in the worst case and the amortized time cost. Inserting or deleting an element from an array can be regarded as a meta operation.

Solution. 1. Insertion Algorithm

Suppose the data structure currently holds n elements. To insert a new element (the $(n+1)$ -th element), we perform the following steps:

- (a) Find the smallest index i such that array A_i is currently **empty**.
- (b) Because A_i is the first empty array, all preceding arrays $A_0, A_1, \dots, A_{i-1}$ must be **full**.
- (c) **Delete** all elements from $A_0, A_1, \dots, A_{i-1}$. The total number of elements extracted is:

$$\sum_{j=0}^{i-1} 2^j = 2^i - 1$$

- (d) Combine these $2^i - 1$ extracted elements with the 1 new element to form a new set of exactly 2^i elements.
- (e) **Insert** these 2^i elements into array A_i , making it full.

2. Worst-Case Time Cost

The worst-case scenario occurs when all existing arrays are full. This happens when $n = 2^k - 1$ (i.e., the binary representation of n consists of k consecutive 1s). To insert the $(n+1)$ -th element, we must find the first empty array, which will be A_k . We must delete all n elements from $A_0, \dots, A_{k-1}$, resulting in n *delete* meta-operations. We then insert the n old elements plus the 1 new element into A_k , resulting in $n+1$ *insert* meta-operations.

Total worst-case cost: $n + (n+1) = 2n+1$ meta-operations.

Worst-case time complexity: $\mathcal{O}(n)$.

3. Amortized Time Cost (Aggregate Analysis)

We can analyze the amortized cost by calculating the total cost of continuously inserting n elements starting from an empty data structure. We count the total number of *Insert* and *Delete* meta-operations separately.

- **Total Insertions:** An element is inserted into A_i every time A_i is filled. This occurs once every 2^i operations. When it happens, exactly 2^i elements are inserted.

$$\text{Total inserts for } A_i \leq \frac{n}{2^i} \times 2^i = n$$

Since there are at most $\approx \log_2 n$ arrays, the total number of insert operations across all arrays is bounded by $n \log_2 n$.

- **Total Deletions:** An array A_i is emptied every time it contributes to a "carry-over" to fill A_{i+1} . This occurs once every 2^{i+1} operations. When it happens, 2^i elements are deleted from A_i .

$$\text{Total deletes for } A_i \leq \frac{n}{2^{i+1}} \times 2^i = \frac{n}{2}$$

Summing this over all $\approx \log_2 n$ arrays, the total number of delete operations is bounded by $\frac{n}{2} \log_2 n$.

Final Amortized Cost:

The total number of meta-operations over n successive insertions is the sum of all inserts and deletes:

$$\text{Total Operations} \leq n \log_2 n + \frac{n}{2} \log_2 n = 1.5n \log_2 n$$

To find the **amortized cost per operation**, we divide the total cost by n :

$$\text{Amortized Cost} \leq \frac{1.5n \log_2 n}{n} = 1.5 \log_2 n$$

Amortized time complexity: $\mathcal{O}(\log n)$. □

2. **(DFS&BFS)** Let $G = (V, E)$ be a graph and s be a vertex such that there is a path from s to each $u \in V$. We say G is a **good graph** if there exists a tree $T = (V, E)$ that share the same vertex set V with G such that T is both a depth-first search tree and a breadth-first search tree.

- (a) If G is an undirected graph, prove that G is a **good graph** if and only if G is a tree.
- (b) If G is a **good directed acyclic graph**, prove or disprove that the vertex set V can be sorted in an array L such that L is both an non-decreasing order of the distances from s and a topological order.

Solution. (a) i. **Sufficiency ($\Leftarrow$):** If G is a tree, obviously its DFS tree is the same as its BFS tree, namely it is a **good graph**.

Necessity ($\Rightarrow$): We prove by contradiction. Assume G is a good graph but not a tree, so G contains a cycle $C = (x, u_1, u_2, \dots, u_k, x)$ where $k \geq 2$. Let x be the vertex on C with minimum depth in T .

- **Key lemma:** In an undirected DFS, all non-tree edges are back edges (no cross edges exist). This is because when edge $\{u, v\}$ is explored, one endpoint is already an ancestor of the other.
- **BFS perspective:** Since $\{x, u_1\}$ and $\{x, u_k\}$ are edges in G and x has minimum depth on C , both u_1 and u_k must be at depth exactly $d(x) + 1$.
- **DFS perspective:** Since x has minimum depth on C , all other cycle vertices are descendants of x . By the key lemma, the path $x \rightarrow u_1 \rightarrow \dots \rightarrow u_k$ consists of tree edges, forcing $u_1, u_2, \dots, u_k$ into a strict ancestor-descendant chain. Thus $d(u_k) \geq d(u_1) + 1 = d(x) + 2$.

Contradiction! BFS requires $d(u_k) = d(x) + 1$, while DFS requires $d(u_k) \geq d(x) + 2$. Therefore G must be a tree.

- (b) **Disprove.** The statement is **false**. We can disprove it by constructing a counterexample. Let $G = (V, E)$ be a directed acyclic graph where:

- $V = \{s, A, B, C\}$

- $E = \{(s, A), (s, C), (A, B), (B, C)\}$

1. Prove G is a "good directed acyclic graph":

Let $T = (V, E')$ be a spanning tree with $E' = \{(s, A), (s, C), (A, B)\}$.

- T is a **BFS tree**: Starting from s , the queue discovers A and C at level 1. Then from A , it discovers B at level 2. When exploring B 's neighbors, C is already visited. Thus, the non-tree edge (B, C) is safely ignored, and the BFS tree is exactly T .
- T is a **DFS tree**: Suppose the DFS traversal visits neighbor C before A . The path goes $s \rightarrow C$. C has no outgoing edges, so DFS backtracks to s . Next, it visits $s \rightarrow A \rightarrow B$. At vertex B , the only neighbor is C . Since C has already been completely explored, (B, C) is classified as a valid **cross edge** in the directed DFS. The resulting DFS tree is exactly T .

Since T is both a DFS tree and a BFS tree, G is a good graph. Also, G clearly contains no cycles, so it is a good DAG.

2. Prove the contradiction in array L :

The shortest path distances from s are:

$$\text{dist}(s, s) = 0, \quad \text{dist}(s, A) = 1, \quad \text{dist}(s, C) = 1, \quad \text{dist}(s, B) = 2$$

Assume for contradiction that there exists an array L satisfying both conditions:

- Because L is a **topological order** and there exists a directed edge $(B, C) \in E$, vertex B must appear strictly before vertex C in L .
- Because L is in a **non-decreasing order of distances**, the fact that B appears before C implies $\text{dist}(s, B) \leq \text{dist}(s, C)$.

However, $\text{dist}(s, B) = 2$ and $\text{dist}(s, C) = 1$, which gives $2 \leq 1$. This is a contradiction!

Therefore, such an array L does not necessarily exist for a good DAG.

□

3. (Path) You are given a directed graph G , and a start vertex u . Every vertex in the graph contains a non-negative number of points you can collect on the first time you visit the vertex (if you visit the vertex again you get no points the second time, but there's no penalty). You would like to figure out how many points you could collect in the graph. Design algorithms respectively if

- G is strongly connected. (hint: you do not have to find the exact path, just the value.)
- G is a DAG.
- G is any directed graph.

For (a) and (b), just explain your idea. For (c), give full algorithm along with pseudocode.

Solution. (a) In a strongly connected graph, there is a path between every pair of vertices. Since there is no penalty for revisiting vertices, we can start at u and navigate through the graph to visit every single vertex. Therefore, the maximum points we can collect is simply the sum of the points of all vertices in V .

- (b) Since there are no cycles, we cannot revisit vertices. The problem translates to finding the maximum-weight path starting from u . We can solve this efficiently using **Topological Sorting** and **Dynamic Programming**:

- First, topologically sort all vertices in the DAG.

- Let $DP[v]$ be the maximum points collected on a path from u to v . Initialize $DP[u] = \text{points}[u]$ and $DP[v] = -\infty$ for all other vertices.
- Iterate through the vertices in topological order. For each reachable vertex x and its outgoing edge (x, y) , update the maximum points for y :

$$DP[y] = \max(DP[y], DP[x] + \text{points}[y])$$

- The maximum points we can collect in the graph is the maximum value in the entire DP array.

(c) **Idea:** As we know in 3a we can find all SCCs and condense each SCC into a single "super-vertex". The weight of this super-vertex is the sum of the points of its constituent nodes. After this graph condensation, the resulting graph G^{DAG} is strictly a DAG. We then locate the super-vertex containing the start node u and apply 3b on G^{DAG} to find the maximum collected points.

Algorithm Pseudocode:

Algorithm 1: Maximum Points in Arbitrary Directed Graph

Input: Directed graph $G = (V, E)$, start vertex u , array points
Output: Maximum points collectable starting from u

```

1  $SCC\_map, num\_SCCs \leftarrow \text{FindSCC}(G)$  ; //  $SCC\_map[v]$  gives the SCC ID for  $v$ 
2 Initialize  $W[1 \dots num\_SCCs]$  with all 0s;
3 Initialize  $G^{DAG}$  as an empty graph with  $num\_SCCs$  vertices;
4 for each vertex  $v \in V$  do
5    $S_v \leftarrow SCC\_map[v]$ ;
6    $W[S_v] \leftarrow W[S_v] + \text{points}[v]$ ;
7   for each neighbor  $next\_v$  of  $v$  in  $G$  do
8      $S_{next} \leftarrow SCC\_map[next\_v]$ ;
9     if  $S_v \neq S_{next}$  and  $(S_v \rightarrow S_{next}) \notin G^{DAG}$  then Add directed edge
       $(S_v \rightarrow S_{next})$  to  $G^{DAG}$  ;
10  $TopoOrder \leftarrow \text{TopologicalSort}(G^{DAG})$ ;
11 Initialize  $DP[1 \dots num\_SCCs]$  with  $-\infty$ ;
12  $start\_S \leftarrow SCC\_map[u]$ ;
13  $DP[start\_S] \leftarrow W[start\_S]$ ;
14  $max\_pts \leftarrow DP[start\_S]$ ;
15 for each super-vertex  $S$  in  $TopoOrder$  do
16   if  $DP[S] \neq -\infty$  then
17     for each neighbor  $next\_S$  of  $S$  in  $G^{DAG}$  do
18       if  $DP[S] + W[next\_S] > DP[next\_S]$  then
19          $DP[next\_S] \leftarrow DP[S] + W[next\_S]$ ;
20    $max\_pts \leftarrow \max(max\_pts, DP[S])$ ;
21 return  $max\_pts$ ;

```

□

4. **(Flow&Cut)** Given a bipartite graph $G = (V, E)$ with n vertices and m edges, we want to find the largest **independent set** I , i.e., a subset $I \subseteq V$ such that no two vertices in I are adjacent in G . One way to solve the problem is to construct a flow network (a directed graph) $G = (V \cup \{s, t\}, E')$, where s is the source and t is the sink. Let V_L and V_R denote the left

and right side of V in G . For each $u \in V_L$, we add the directed edge (s, u) to E' . For each $v \in V_R$, we add the directed edge (v, t) to E' . For each $u, v \in E$ with $u \in V_L$ and $v \in V_R$, we add the directed edge (u, v) to G' . All edges have capacity 1.

- (a) Prove that if there is an independent set in G of size k , then there is an (s, t) -cut in G' of capacity $n - k$.
- (b) Conversely, prove that if there is an (s, t) -cut in G' of capacity $n - k$, then there is an independent set in G of size k .
- (c) Conclude that there is a polynomial-time algorithm to compute a largest independent set in G .

Solution. (a) Let $I \subseteq V$ be an independent set of size k . By the definition of an independent set, no two vertices in I share an edge. Let $C = V \setminus I$. Since I contains no edges, every edge in G must have at least one endpoint in C . Thus, C is a **vertex cover** of size $n - k$. We construct a cut (S, T) in the flow network G' as follows:

- Put the source $s \in S$, and the sink $t \in T$.
- For vertices in V_L : if $u \in I$, assign $u \in S$; if $u \in C$, assign $u \in T$.
- For vertices in V_R : if $v \in I$, assign $v \in T$; if $v \in C$, assign $v \in S$.

The capacity of the cut $c(S, T)$ is the number of edges directed from S to T . Let's analyze the three types of edges in G' :

- **Edges (s, u) :** This edge crosses from S to T if and only if $u \in T$. By our construction, this means $u \in V_L \cap C$. The number of such edges is $|V_L \cap C|$.
- **Edges (v, t) :** This edge crosses from S to T if and only if $v \in S$. By our construction, this means $v \in V_R \cap C$. The number of such edges is $|V_R \cap C|$.
- **Edges (u, v) for $u \in V_L, v \in V_R$:** This edge crosses from S to T if and only if $u \in S$ and $v \in T$. By our construction, this requires both $u \in I$ and $v \in I$. However, since I is an independent set, there are absolutely no edges between any vertices in I . Thus, no intermediate edges cross from S to T .

Summing these up, the total capacity of the cut is:

$$c(S, T) = |V_L \cap C| + |V_R \cap C| = |C| = n - k$$

Thus, an (s, t) -cut of capacity $n - k$ exists.

- (b) Let (S, T) be an (s, t) -cut with capacity $n - k$. The capacity is the total number of edges directed from S to T . These cut edges can be of three types: (s, u) where $u \in V_L \cap T$, (v, t) where $v \in V_R \cap S$, or intermediate edges (u, v) where $u \in V_L \cap S$ and $v \in V_R \cap T$. We construct a set of vertices $C' \subseteq V$ by picking one endpoint for each cut edge from S to T :

- For every cut edge (s, u) , we add u to C' .
- For every cut edge (v, t) , we add v to C' .
- For every cut edge (u, v) where $u \in V_L \cap S$ and $v \in V_R \cap T$, we add u to C' .

Since the capacity is $n - k$, we pick at most one vertex per cut edge, meaning $|C'| \leq n - k$.

We claim that C' is a **vertex cover** for G . Assume for contradiction that there exists an edge $(x, y) \in E$ (with $x \in V_L, y \in V_R$) that is not covered by C' . This means $x \notin C'$ and $y \notin C'$.

- Since $x \notin C'$, the edge (s, x) was not cut, so x must be in S .

- Since $y \notin C'$, the edge (y, t) was not cut, so y must be in T .
- But if $x \in S$ and $y \in T$, the intermediate edge (x, y) crosses from S to T . According to our rule, we should have added x to C' for this cut edge, contradicting $x \notin C'$.

Therefore, C' must cover all edges in G .

Let $I = V \setminus C'$. Since C' is a vertex cover, its complement I must be an independent set. The size of I is:

$$|I| = n - |C'| \geq n - (n - k) = k$$

Since we have an independent set of size at least k , we can simply select exactly k vertices from it to obtain an independent set of size k .

- (c) From the proofs above, finding the **largest** independent set (maximizing k) is mathematically equivalent to finding the **minimum** (s, t) -cut (minimizing $n - k$) in the directed flow network G' .

According to the Max-Flow Min-Cut Theorem, the minimum capacity of an (s, t) -cut equals the maximum flow from s to t . The algorithm proceeds as follows:

- Construct the flow network G' from the bipartite graph G , assigning a capacity of 1 to all edges.
- Run a polynomial-time maximum flow algorithm to find the max flow and the residual graph.
- Perform a Breadth-First Search (BFS) from s in the residual graph to find the set of reachable vertices, which forms the set S of the minimum (s, t) -cut. T is the rest of the vertices.
- Identify the vertex cover C' by extracting $V_L \cap T$ and $V_R \cap S$.
- Return the complement $V \setminus C'$ as the largest independent set.

Constructing the graph, computing the max flow, and extracting the cut all run in polynomial time. Thus, there is a polynomial-time algorithm to compute the largest independent set in a bipartite graph.

□